

ExactPoly: A Java Library for Exact Symbolic Polynomial Arithmetic over the Rationals

Trinsit W.

Independent Researcher

<https://github.com/trinsitw>

May 2026

Abstract

We describe EXACTPOLY, a Java library for exact symbolic arithmetic on univariate and multivariate polynomials with rational coefficients. The library emerged as a reusable artifact from a study of a recursive identity for power sums [4], and is general-purpose in design. All polynomial coefficients are represented as exact rational numbers, ensuring that no rounding error is introduced across recursive computations. A key capability is *polynomial composition*: the substitution of a variable with another polynomial, yielding a new polynomial in closed symbolic form. We describe the architecture of the five core classes, their design properties, and demonstrate the library through worked examples drawn from combinatorics, number theory, and symbolic computation.

Keywords: ExactPoly, symbolic computation, polynomial arithmetic, exact arithmetic, rational numbers, computer algebra, Java.

1 Introduction

Exact symbolic polynomial computation is a foundational capability in computer algebra. Systems such as Mathematica and SymPy provide polynomial arithmetic as part of larger ecosystems, but standalone, dependency-free implementations in Java — particularly those that maintain exact rational arithmetic throughout — are less common.

This library, named **ExactPoly**, was developed alongside a study of a recursive identity for the power sum $S(k, n) = \sum_{i=1}^n i^k$ [4]. During that work it became apparent that the polynomial manipulation routines had value independently of the problem that motivated them, and warranted separate documentation.

The library provides five core classes:

- **RationalNumber**: arbitrary-precision rational arithmetic,
- **IndeterminateExponent**: a variable–exponent pair,
- **MultivariateMonomial**: a rational coefficient times a product of variables,
- **MultivariatePolynomial**: a sum of multivariate monomials,
- **RationalPolynomial**: a dense univariate polynomial with division.

No floating-point arithmetic is introduced at any stage. All intermediate and final results are exact.

2 Library Architecture

Figure 1 shows the relationships among the five core classes. `RationalNumber` is the foundational value type used throughout; `IndeterminateExponent` and `MultivariateMonomial` build upward toward `MultivariatePolynomial`, while `RationalPolynomial` provides a parallel univariate track optimised for division.

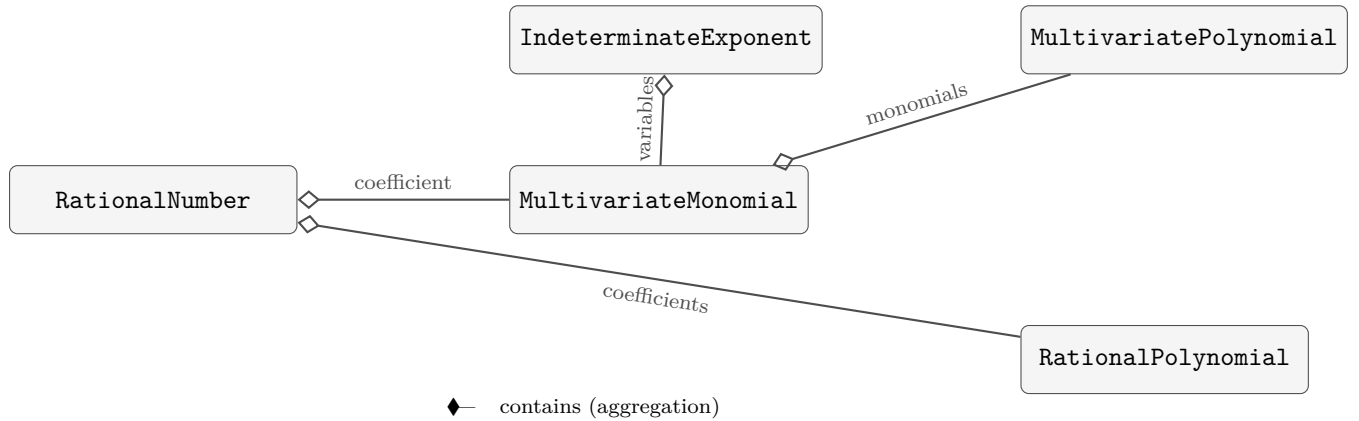


Figure 1: Class relationships in `ExactPoly`. Diamond-end arrows indicate aggregation: the tail class holds instances of the head class. `MultivariatePolynomial` and `RationalPolynomial` provide parallel tracks for multivariate and univariate computation respectively.

2.1 `RationalNumber`

The foundational type. Internally stores numerator and denominator as `BigInteger`, reduced to lowest terms at construction time. The sign is normalised so the denominator is always positive. The four arithmetic operations and integer exponentiation are supported.

Key design properties:

- *Immutable value type.* Every operation returns a new instance; no mutation occurs.
- *Automatic reduction.* The GCD is computed and divided out at construction, eliminating the need for a separate normalisation step.
- *Equality by value.* `equals` implements the mathematical criterion $a/b = c/d \iff ad = bc$.

Constants `ZERO` and `ONE` are provided as static fields.

2.2 `IndeterminateExponent`

A pair (`char indeterminate`, `int exponent`) representing a single variable raised to a positive integer power, e.g. x^3 or n^2 . Indeterminates are restricted to lowercase letters `a–z`. The class implements `Comparable<IndeterminateExponent>` by variable name, enabling canonical ordering within monomials.

2.3 `MultivariateMonomial`

A product of a `RationalNumber` coefficient and a list of `IndeterminateExponent` factors, representing terms such as $\frac{3}{2}x^2y^3$. The constructor normalises the variable list into sorted canonical order and collapses the zero-coefficient case to an empty variable list.

Multiplication of two monomials merges their variable lists by grouping on the variable name and summing exponents. The `indeterminateExponentList()` accessor returns a defensive copy, preventing external mutation of internal state.

2.4 MultivariatePolynomial

A sum of `MultivariateMonomial` terms. The constructor automatically combines like terms (monomials sharing the same variable signature) and eliminates zero terms, maintaining a canonical form at all times.

Supported operations include `add`, `subtract`, `multiply`, `pow`, `evaluate`, and `commonDenominatorForm`. The most distinctive operation is `compose`, described separately below.

2.5 Polynomial Composition

The `compose(chart, MultivariatePolynomials)` method substitutes every occurrence of the indeterminate t in a polynomial p with a polynomial s , returning the result as a new `MultivariatePolynomial`. Formally, if p is a polynomial in variables including t , then

$$p.\text{compose}(t, s) = p[t \mapsto s],$$

which is itself a polynomial in the remaining variables and the variables of s .

This elevates polynomials from passive data structures to *first-class functions*: instead of evaluating p at a number, `compose` evaluates p at another polynomial and returns an exact symbolic result. The operation is safe because all types in `ExactPoly` are immutable — the substitution produces a fresh object and leaves p and s unchanged.

Role in power-sum computation. `compose` is the mechanism that makes the recursive power-sum algorithm in [4] work. The identity requires evaluating $S(k-2, i-1)$, i.e. the already-derived polynomial for $S(k-2, n)$ with n replaced by $i-1$. This is expressed directly as:

$$S(k-2, i-1) = \text{Sk2}.\text{compose}('n', \text{iMinus1})$$

where `iMinus1` is the polynomial $i-1$. The result is a symbolic polynomial in i that can then be multiplied, summed, and processed further — all without leaving the symbolic domain. Without `compose`, this step would require either a numeric approximation or a fundamentally different algorithm.

Broader applicability. Beyond the motivating application, `compose` enables a range of symbolic tasks: computing $p(n+1) - p(n)$ to verify telescoping identities, constructing shifted polynomials $p(x-a)$ for change-of-variable arguments, and building higher-order expressions by functional substitution.

2.6 RationalPolynomial

A dense univariate polynomial stored as a coefficient array indexed by degree. Supports `add`, `subtract`, `multiply`, `pow`, `scalar divide`, `evaluate` via Horner’s method, and polynomial long division returning `[quotient, remainder]`.

`RationalPolynomial` is suited to problems where a single variable is known in advance and division is required; `MultivariatePolynomial` is suited to problems involving several variables or polynomial composition.

3 Design Properties

Two properties follow directly from the implementation and are worth stating explicitly.

Canonical form. Every `MultivariatePolynomial` instance is in canonical form: like terms are combined, zero terms are absent, and monomials are sorted by total degree in descending

order. This is enforced in the constructor via `groupingBy` on the variable signature, followed by filtering and sorting. Since all operations return new instances through the same construction path, the invariant is preserved throughout. As a consequence, two instances represent the same polynomial if and only if they are structurally equal, and `equals` is a simple set comparison on monomials.

Immutability and composition safety. Every type in `ExactPoly` is immutable: `RationalNumber`, `IndeterminateExponent`, `MultivariateMonomial`, and `MultivariatePolynomial` all return new instances from every operation, and all list accessors return defensive copies. This is not merely a stylistic choice — it is a prerequisite for `compose` to be correct. Polynomial composition substitutes a variable with a polynomial and uses both the original and the substitution in subsequent reductions. If any operand were mutable, a modification made during the substitution could silently corrupt a memoised intermediate result, producing incorrect output that is difficult to detect and reproduce. Immutability guarantees that `compose` is referentially transparent: the same inputs always produce the same output, regardless of what other computations are in progress.

No `RationalNumber` operation introduces rounding error. All arithmetic is performed on `BigInteger` numerators and denominators, with GCD reduction at each construction step. Integer overflow is impossible since `BigInteger` is unbounded. This property is what makes the library suitable for deep recursive computations such as the power-sum derivation in [4], where floating-point error would accumulate across many levels.

4 Example Applications

4.1 Verifying the Closed Form for $S(2, n)$

The classical formula $S(2, n) = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n$ can be constructed and numerically verified:

```

1 MultivariatePolynomial S2 = new MultivariatePolynomial(
2     new MultivariateMonomial(new RationalNumber(1, 3), 'n', 3),
3     new MultivariateMonomial(new RationalNumber(1, 2), 'n', 2),
4     new MultivariateMonomial(new RationalNumber(1, 6), 'n', 1)
5 );
6
7 // S(2,4) = 1 + 4 + 9 + 16 = 30
8 System.out.println(S2.evaluate(BigInteger.valueOf(4))); // 30

```

4.2 Polynomial Composition: Telescoping and Recursive Substitution

`compose` treats a polynomial as a function and evaluates it at another polynomial, returning an exact symbolic result. Two uses are illustrated here.

Telescoping verification. A closed-form candidate $p(n)$ for $S(k, n)$ must satisfy $p(n+1) - p(n) = (n+1)^k$. For $S(1, n) = \frac{1}{2}n^2 + \frac{1}{2}n$, we expect $p(n+1) - p(n) = n+1$:

```

1 MultivariatePolynomial p = new MultivariatePolynomial(
2     new MultivariateMonomial(new RationalNumber(1, 2), 'n', 2),
3     new MultivariateMonomial(new RationalNumber(1, 2), 'n', 1)
4 );
5
6 // Substitute n -> (n + 1)
7 MultivariatePolynomial nPlus1 = new MultivariatePolynomial(
8     new MultivariateMonomial(new RationalNumber(1), 'n', 1),
9     new MultivariateMonomial(new RationalNumber(1))
10 );
11

```

```

12 // p(n+1) - p(n) should equal (n+1)
13 MultivariatePolynomial diff = p.compose('n', nPlus1).subtract(p);
14 System.out.println(diff); // (1)(n^1) + (1) i.e. n+1 [correct]

```

Recursive substitution in power-sum derivation. The recursive identity for $S(k, n)$ [4] requires forming $S(k-2, i-1)$: the polynomial for the lower-order sum, with n replaced by $i-1$. `compose` produces this in one call, returning a symbolic polynomial in i ready for further arithmetic:

```

1 // Sk2 is the already-derived MultivariatePolynomial for S(k-2, n)
2 // Substitute n -> (i - 1)
3 MultivariatePolynomial iMinus1 = new MultivariatePolynomial(
4     new MultivariateMonomial(new RationalNumber(1), 'i', 1),
5     new MultivariateMonomial(new RationalNumber(-1))
6 );
7
8 MultivariatePolynomial Sk2AtIMinus1 = Sk2.compose('n', iMinus1);
9 // Result: S(k-2, i-1) as a polynomial in i, exact, ready to sum

```

Without `compose`, this step would require a numeric approximation or a fundamentally different algorithm. The result remains fully symbolic and is passed directly into the next stage of the recursion.

4.3 Polynomial Long Division

Dividing $x^3 - x$ by $x - 1$ to obtain quotient $x^2 + x$ and remainder 0:

```

1 // Coefficients in ascending degree order
2 RationalPolynomial num = new RationalPolynomial(
3     RationalNumber.ZERO, new RationalNumber(-1),
4     RationalNumber.ZERO, RationalNumber.ONE
5 );
6 RationalPolynomial den = new RationalPolynomial(
7     new RationalNumber(-1), RationalNumber.ONE
8 );
9
10 RationalPolynomial[] result = num.divide(den);
11 System.out.println("Quotient: " + result[0]); // x^2+x
12 System.out.println("Remainder: " + result[1]); // 0

```

4.4 Multivariate Expansion: $(x + y)^3$

```

1 MultivariatePolynomial xPlusY = new MultivariatePolynomial(
2     new MultivariateMonomial(RationalNumber.ONE, 'x', 1),
3     new MultivariateMonomial(RationalNumber.ONE, 'y', 1)
4 );
5
6 System.out.println(xPlusY.pow(3));
7 // (1)(x^3) + (3)(x^2)(y^1) + (3)(x^1)(y^2) + (1)(y^3)

```

5 Related Work

Several open-source Java libraries address symbolic polynomial computation, but each targets a different point in the design space.

JAS (Java Algebra System) is a comprehensive library for algebraic computation in Java,

built on generic types, with support for commutative and solvable polynomials, Gröbner bases, and an experimental factorisation package [5]. JAS is large, depends on several external libraries, and exposes an API designed for breadth of algebraic structures rather than simplicity of use.

Rings is a high-performance Java/Scala library focused on polynomial rings, supporting multivariate polynomial GCD, factorisation, linear system solving, and Gröbner bases [6]. Its design priority is computational speed for challenging real-world problems; it is not intended as a lightweight or pedagogically transparent component.

Symja is a full computer algebra system in Java, providing arbitrary-precision integers and rationals, symbolic differentiation and integration, equation solving, pattern matching, and polynomial functions, with Mathematica-like syntax and notebook interfaces [7]. Its scope is substantially broader than polynomial arithmetic alone.

The library described in this paper occupies a different point in the design space from all of the above. It is minimal (five classes), has no external dependencies beyond the Java standard library, and was designed to be structurally transparent — the algebraic intent of each class is directly legible from its implementation. None of the libraries above were designed with this combination of constraints.

6 Limitations and Future Work

The current implementation prioritises correctness and structural clarity over computational performance. Three directions for future development are noted.

1. *Performance.* For polynomials with many variables and high degrees, the use of `ArrayList` and stream-based operations introduces overhead. A sparse hash-map representation indexed by variable signature would improve performance in high-degree cases.
2. *Missing operations.* Greatest common divisor of polynomials, resultants, and factorisation are not currently implemented.
3. *Variable names.* Indeterminates are restricted to single lowercase letters (`a–z`). Extending to multi-character identifiers would improve readability for applied problems.

7 Conclusion

We have described EXACTPOLY, a Java library for exact symbolic polynomial arithmetic. Its design emphasises correctness, immutability, and canonical representation. The library requires no external dependencies beyond the Java standard library, and has been used in practice to derive closed-form polynomial expressions for $S(k, n)$ for k up to 100 [4].

The source code is available at <https://github.com/trinsitw/java>.

Acknowledgement

The mathematical ideas and Java library described in this paper are entirely the author’s own work, written by hand without the use of AI code generation tools. AI assistance (Claude, Anthropic) was used for manuscript preparation and L^AT_EX typesetting.

References

- [1] J. von zur Gathen and J. Gerhard, *Modern Computer Algebra*, 3rd ed., Cambridge University Press, 2013.

- [2] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed., Addison-Wesley, 1997.
- [3] G. E. Collins, “Subresultants and reduced polynomial remainder sequences,” *Journal of the ACM* 14(1), 128–142, 1967.
- [4] Trinsit W., “A Recursive Identity for Power Sums: An Alternative to Faulhaber’s Formula,” Independent Research, 2026. <https://github.com/trinsitw/java>
- [5] H. Kredel, “Formally verified mathematics with JAS and Coq,” in *Proc. International Workshop on Programming Languages for Mechanized Mathematics Systems*, 2008. <https://krum.rz.uni-mannheim.de/jas/>
- [6] S. Poslavsky, “Rings: An efficient Java/Scala library for polynomial rings,” *Computer Physics Communications* 235, 400–413, 2019.
- [7] A. Kramer, *Symja — Computer Algebra Language and Symbolic Math Library*, GitHub, 2023. https://github.com/axkr/symja_android_library